# CE152: Object-Oriented Programming

## Data types, Inheritance

Dr. Sebastian Halder

CSEE

2026

# Data types

# Primitives and Objects

- In Java there are two types of variables: *primitives* and *Objects*
- *Primitives* are the basic building blocks of classes
- *Classes* are made of these building blocks or other *classes*
- *Objects* are instances of classes
- Numerous *Classes* are built into the Java Class Library (JCL)
- ***VERY IMPORTANT:*** In Java most lines end with a semicolon and you *must declare the type* of the variable!
- Of course there are exceptions but basically all declarations and expressions end with a semicolon

```
// By the way, this is how you declare a comment in Java
int i = 5; // primitive integer
Integer j = new Integer(10); // Object integer
j.toString(); // Objects can have methods
              // They are based on a class!
```

# Primitives are more memory efficient

```
/* You may now be wondering why not always use Objects?
The primitive data types are much more memory efficient!
So use them when you can.

This by the way is how you declare a multiline
comment in Java. */
int x = 1; // this uses 32 bits...

Integer y = new Integer(1); // this uses 128 bits!
```

# Car example

```
class Car{
  /*
  1. List variables
  */
  int speed;
  String colour;

  /*
  2. Define constructor
  */

  /*
  3. Add methods
  */
}
```

# Examples of primitives

```
byte b = 10;
short s = 20_000;
int i = 300_000_000;
long l = 400_000_000_000_000L;
float f = 5.123f;
double d = 6.456789012345678;
char c = 's';
boolean bb = true;
```

- We can use _ to make longer numbers easier to read
- The _ has no effect on the value as such

# Remark on floating point numbers

- What is important to know that all data type of the same size are able to represent the same number of values
- E.g. `int` can represent as many values as `float` (both are 32 bit)
- What differs is the range and the size of the intervals
- The integer data types all represent all integers in their range
- The floating point data types represent the numbers in their range with a certain precision
- Interestingly the precision varies with the number
- Some integers can actually not be represented by `float` (e.g.: 16_777_217) or `double` (e.g.: 9_007_199_254_740_993)
- If we assign them to `float`/`double` the value will be rounded to the next possible value

# Why do `byte`, `short`, `int` and `long` exist?

- In theory we could use `long` anytime we want to store integers to be on the safe side
- It is *good practice* to always use the data type that covers the expected data range
- The impact on performance is very small though with the number of variables we will be dealing with in CE152
- The information that you provide on what you expect the data type to contain may be the most important aspect
- ***Remember***: you do not only write code to achieve a particular goal but to also share it with other people which makes ***readability*** very important
- The data types `byte` and `short` are more important for embedded systems
- The same is true for `float` and `double`

# Objects

# Objects and classes

- In Object-Oriented Programming we create building blocks
- These building blocks consist of ***templates or blueprints: the classes***
- When we instantiate classes we create ***objects*** using the ***constructor*** of that class
- This is analogous to objects in reality where we have classes, e.g. chair, and can have multiple objects of that type
- Please refer to the video on *Objects* for further information
- In the following we will look at the built-in objects provided by the JDK

# Wrappers

- The primitives we discussed earlier all of have analogous wrapper classes
- These are Objects containing the primitive
- The reason for the existence of wrapper classes is that many principles we use in OOP can only be applied to Objects
- For example, there are very useful classes to store multible variables, so-called collections, that can only contain Objects
- The reason that they contain only Objects is that all variables contained in them must implement certain methods, e.g. for sorting
- ***Please remember: Only Objects have methods***
- ***Please remember: Primitives are not Objects, Wrappers are***

# Instantiating wrapper Objects

- We can use the `new` operator or methods dedicated for creating new Wrapper Objects from primitives

```
Integer wrapperOne = new Integer(5);
Integer wrapperTwo = Integer.valueOf(5);
```

- We can convert the wrapper back to a primitive

```
int i = wrapperOne.intValue();
```

# Use of Wrappers

- We can determine the minimum and maximum values of the primitives used variables contained in the wrapper classes:

```
Double.TYPE; Double.SIZE;
Double.MIN_VALUE; Double.MAX_VALUE;
Integer.TYPE; Integer.SIZE;
Integer.MIN_VALUE; Integer.MAX_VALUE;
```

- We can use Wrappers to convert text to numerical data (this is very useful when reading data from files):

```
int i = Integer.parseInt("152");
double d = Double.parseDouble("152.152");
```

- As mentioned before Wrappers allow the use of Java Collections (more on that in another lesson)!

# Important classes for CE152

- There are about 4500 classes in the JDK
- For now we will look at some useful examples: `String` (and related classes) and `System`
- `String` based operations are quite common and something that is easily possible using primitives
- Essentially `String` is an class that can hold multiple (or just one) `char`

# Examples of creating a String

```
String s1 = String.valueOf('c');
String s2 = new String('c');
String s3 = new String("CE152");
String s4 = "CE152";
String s5 = new String('CE152'); //NOT POSSIBLE!!
```

# Using String

```
String s1 = String.valueOf("abcde");
char c1 = s1.charAt(0); // a; indices start with 0
char c2 = s1.charAt(2); // c
s1 = s1.toUpper(); //ABCDE
s1 = s1.toLower(); //abcde
int i = s1.length(); //5
String s2 = String.valueOf("ABCDE");
boolean b1 = s1.equals(s2); // false
boolean b2 = s1.equalsIgnoreCase(s2) //true
```

# Remarks on concatenation

- `String` can be concatenated with another `String`
  - `"AB" + "C"` becomes `"ABC"`
- `String` can be concatenated with `int`
- It is important to note that **operator precedence** for + is from left to right
- How will this affect the output of the following code?

```
String s1 = "40 + 2 = " + 40 + 2;
String s2 = "40 + 2 = " + (40 + 2);
String s3 = 40 + 2 + " = " + 40 + 2;
```

# StringBuilder

- Sometimes we want to construct a `String` in steps
- For this we can use `StringBuilder` (this is another class!)

```
StringBuilder sb = new StringBuilder();
sb.append('a');
sb.append('b');
sb.append('c');
String s = sb.toString();
```

- We use `StringBuilder` becauser operations like this actually create a `StringBuilder` in the background in each step which is slower

```
String s = new String();
s += 'a';
s += 'b';
s += 'c';
```

# Printing to the command line

- In Java we use a class called `System` for printing to the command line
- This class has a class variable called `out` of type `PrintStream` which is `static` (this means we can just use it without assigning it to a variable)
- This variable has a method called `println()`
- `System` has some other variables has well that we will learn about later
- This method is what we use to print in Java

```
System.out.println("Hello CE152");
```

# Import statements

- The `import` statements make classes accessible without having to write the full names
- Must be placed before any class declarations
- With the star symbol we can import all classes from one package
  - If this leads to a name clash we have to import individual classes instead
- We can import our packages to other packages in the same way

# Importing packages (examples)

```
import java.io.File;
import java.io.FileNotFoundException;
import java.io.PrintStream;
import java.util.Arrays;
import java.util.Scanner;
```

# Creating packages in IntelliJ

- In the Project View, select the “src” folder
- Then click on “File -> New -> Package” and create a new package called e.g. lab1
- Select the “lab1” package in the project view, then click on “File -> New -> Classes” and create new classes
- This class is created in then package lab1

# Casting

# Changing the data type

- We heard about casting e.g. when we cast a char to an int and vice versa:

```
// Will the variable b contain a letter or a number?
int a = (int)'b';
char b = (char)a;
```

- We will learn some rules for casting Objects in the video about inheritance!

# Casting primitives

- The types of casting that are distinguished in this case are *widening* and *narrowing* casting
- Remember an int had 32 bits and a double had 64 bits?
- If we cast from *int to double* it is widening and when we cast from *double to int* it is narrowing casting

# Widening vs. narrowing

```
int a = 127;
// This is fine
double b = a;
// This will produce an error (compilation)
int c = b;
// This is fine
int d = (int)b;
```

- A widening cast is automatic, a narrowing cast must be declared
- The reason behind these rules is that casting from a double to an int will cause us to lose information
- The compiler wants to make sure we know this risk (by the way char has 2 bytes (16 bits) so we are narrowing when casting from int to char)

# Inheritance

# Defining a class

- This creates a class called Person
- It does nothing

```
class Person{

}
```

- We could instantiate it using the standard constructor
- The constructor is what follows the new keyword

```
Person myPerson = new Person();
```

# The constructor

- The constructor is the method we use to create an Object based on the class definition
- The default has no parameters and you do not have to declare it like I did below because Java does this for you

```
class Person{
  Person(){
  }
}
```

# Adding member variables

- We add the member variables above the constructor
- We could add them anywhere but it is good practice to put them in one place
- If we do not initialise them they will contain null

```
class Person {
    int age;
    String name;

    Person(){
    }
}
```

# Initialising a Person

- The idea is to make a constructor that initialises our Person to the state you want (the default constructor is now gone!)
- Our Person will be provided with a name we choose and the age will be set to 0

```
class Person {
    int age;
    String name;
    //the this keyword ensures that
    //we assign to the member variable
    Person(String name){
      this.name = name;
      this.age = 0;
    }
}
```

# Adding a method

- When declaring a method we need:
  - The access modifier (can be blank which will be the default (visible in package))
  - The return type (can be void or any primitive or Object)
  - The name (eat, sleep)
  - The parameters (in the (), can be blank or any number of primitives or Objects)

```
void eat(){
}

void sleep(){
}
```

# Adding functionality to the method

- The methods on the previous slide would not do anything when called
- Let's add some output

```
void eat(){
  // This will print to
  // the command line
  System.out.println("omnomnom");
}

void sleep(){
  System.out.println("zzZzzZzzZzz");
}
```

# Adding parameters to the method

- Parameters are added with types and variable names inside the parentheses
- These can now be accessed inside the methods
- Strings can be concatenated with "+"

```
void eat(String food){
    System.out.println(
        "omnomnom " + food);
}

void sleep(int hours){
    System.out.println(
        hours + " hours of zzZzz...");
}
```

# Returning values

- getAge() returns the int age
- This means when we call the method we are provided the age and can process it in some way
- We now increase age by one everytime the Person sleeps (age++ is equal to age = age + 1)

```
int getAge(){
    return age;}

void sleep(int hours){
    System.out.println(
        hours + " hours of zzZzz...");
    // the ++ operator increments by 1
    age++;}
```

# Some additional terms

- We call what is between the $\{\}$ of the method the *body* of the method
- Variables we create inside the method *body* exist only there and are deleted when we leave the method
- They variables inside the *body* are *local variables*
- We may also refer to the method body as a *block*

# The complete Person (1)

```
// We create a class called Person
class Person {
    // With the member variables age and name
    int age;
    String name;
    // In the constructor the name is set by a parameter
    // and the age is set to zero
    Person(String name){
        this.name = name;
        this.age = 0;
    }
    // Our person will eat what we pass as a String
    void eat(String food){
        System.out.println(
                "omnomnom " + food);}
```

# The complete Person (2)

```
    // continued from previous slide
    void sleep(int hours){
        System.out.println(
                hours + " hours of zzZzzZzzzZzz");
        // increases age by one when Person sleeps
        this.age++;
    }

    // tells us the persons age
    int getAge(){
        return this.age;
    }
// This } closes the first curly bracket of the class
}
```

# Deriving Lecturer from Person (1)

```
class Lecturer {
/*
This code will create a new class
called Lecturer with a default
constructor. It will not inherit
anything from Person.
*/

}
```

# Error due to not declaring inheritance

```
class mainExercise1 {
    static void main(String args[])
    {
      // We could now do this:
      Lecturer myLecturer = new Lecturer();

      // The following would result in errors:
      myLecturer.sleep(6);
    }
}
```

# Deriving Lecturer from Person (2)

```
class Lecturer extends Person {
    // Note the extends keyword!
    // Why are we declaring a constructor?
    Lecturer(String name){
        // Note the super keyword!
        super(name);
    }
}
```

# Inheritance in Java

- We use the "extends" keyword to show *inheritance*
- We call the inheriting class *subclass* or *child* (in this case Lecturer is the subclass/child)
- We call the class being inherited from *superclass* or *parent* (in this case Person is the superclass/parent)
- With "super" we can call methods of the *superclass*

# Constructors and inheritance (1)

- You can invoke a parent class constructor by using the keyword `super` followed by appropriate arguments
  - This must be the first statement in the subclass constructor
- If a subclass constructor does not call `super` explicitly, then the parent class must have a zero-argument constructor
  - This is invoked implicitly at the start of the subclass constructor
  - In our case there is no zero-argument constructor thus we must use `super`

```
//zero argument constructor
//no need for super in sub-classes
MyClass(){...}
//constructor with arguments
//if we have only this constructor we need super
MyClass(int x){...}
```

# Constructors and inheritance (2)

- It follows that whenever you create an object of a subclass, a parent class constructor will also be called
- This in turn may trigger a chain of further super-class constructors until the *parent class of all Java classes* is reached which is called `Object`

# Using inherited methods

```
class mainExercise1 {
    static void main(String args[])
    {
      Lecturer myLecturer = new Lecturer("Sebastian");
      // Now this works! What will be the output?
      myLecturer.sleep(6);
    }
}
```

# So why did we do all this?

- Currently Lecturer does exactly what Person does
- We can now add new methods or *override* existing ones
- We can also add new member variables

# The original sleep method

```
...
void sleep(int hours){
    System.out.println(
            hours + " hours of zzZzzZzzzZzz");
    // increases age by one when Person sleeps
    this.age++;
}
...
```

# Overriding a method

```
...
@Override
void sleep(int hours){
    System.out.println("No rest for the lecturer...");
    // ages twice the speed of Person
    this.age = this.age + 2;
}
...
```

- We use `@Override` to tell the compiler that we want to replace the method in the parent class
- It is **not** necessary and if we make no mistakes we can achieve the same effect without `@Override`
- If we make a mistake though the compiler will inform us if there is no parent class method to override

# The two sleep() methods

```
Person myPerson = new Person("Winston");
Lecturer myLecturer = new Lecturer("Sebastian");
myPerson.sleep(12);
System.out.println(myPerson.name + " is " + myPerson.getAge()
+ " undefined units of time old...");
myLecturer.sleep(6);
System.out.println(myLecturer.name + " is " + myLecturer.getAge()
+ " undefined units of time old...");
```

- Output:

```
12 hours of zzZzzZzzzZzz
Winston is 1 undefined units of time old...
No rest for the lecturer...
Sebastian is 2 undefined units of time old...
```

# Adding a new method

```
class Lecturer extends Person{
    Lecturer(String name){
        super(name);
    }

    void sleep(int hours){
        System.out.println("No rest for the lecturer...");
        this.age = this.age + 2;
    }

    double gradeExam(String exam){
        // Math.random() is an rng from 0 to 1
        // What keywords before random() in class def?
        return Math.random()*100;
    }}
```

# Grading exams

```
System.out.println("Your grade is " + Math.round(
  myLecturer.gradeExam("Inheritance...")));
System.out.println("Your grade is " +
  myLecturer.gradeExam("Abstraction..."));
System.out.println("Your grade is " +
  Math.round(myLecturer.gradeExam("Polymorphism...")));
System.out.println("Your grade is " +
  Math.round(myLecturer.gradeExam("...")));
```

- Why does one grade have more decimals?

```
Your grade is 47
Your grade is 6.7928288997513135
Your grade is 73
Your grade is 78
```

# Rules for casting objects

- You can always *upcast* an Object, you can only *downcast* an Object if this is the original type
- We can do this

```
Person myPerson = new Person("Winston");
Lecturer myLecturer = new Lecturer("Sebastian");
myPerson = myLecturer;
```

- But not this!

```
Person myPerson = new Person("Winston");
Lecturer myLecturer = new Lecturer("Sebastian");
myLecturer = myPerson;
```

# Up- and downcasting

- The reason is that the `Person` may not have all the variables of the `Lecturer` while the `Lecturer` will always have all the variables of `Person`
- Upcasting (up the inheritance tree) is always ***safe***
- Sometimes we may need to downcast (down the inheritance tree)
- ***Remember:*** if we cast an object to another type it will still be the original object!
- We can determine the class using `getClass()`

```
String x = "CE152";
Object b = (Object)x;
System.out.println(b.getClass()); // This would output
                                  // class java.lang.String
```

# Summary

# Defining a class in Java

```
class MyClass {}
```

# Defining inheritance in Java

```
class MyClass extends MyParentClass{}
```

# Adding a method to a class

```
class MyClass extends MyParentClass{
  // constructor (does not return anything)
  MyClass(){}
  // method  (returns void or a variable)
  void myMethod(){}
}
```

# The program has to start somewhere

```
//We could also put the main method into MyClass
class MyMain{
  static void main(String[] args){
    MyClass myVariable = new MyClass();
    myVariable.myMethod();
  }
}
```

- ***It is critical that you understand all of the code on this slide and the previous three slides***

# When to use inheritance?

- This is a difficult question
- Primarily inheritance should be used in an additive way
- The class `Lecturer` should be used like the class `Person` plus something extra
- If there are some aspects of the superclass you do not want in your subclass inheritance may not be the best approach
- For example, you could extend your class from `String` instead of adding a variable of type `String`

# Summary

- We learned about one of the four principles of OOP: ***inheritance***
- We use the *class* keyword to declare our class definition
- We use the *extends* keyword to declare inheritance
- We run our code from *static method* called main() from a separate file